

Unit 1.0 — Program Logic Development

1.1 Fundamentals of Algorithms

Algorithm: An algorithm is a finite, well-defined, ordered sequence of steps that takes some input, processes it, and produces a desired output in finite time. Every algorithm must have the following characteristics:

- **Input:** Zero or more well-defined inputs.
- **Output:** At least one well-defined output.
- **Finiteness:** Must terminate after a finite number of steps.
- **Definiteness:** Each step must be precisely and unambiguously defined.
- **Effectiveness:** Each step must be basic enough to be carried out.

Pseudo-code: Pseudo-code is an informal, English-like representation of an algorithm that uses simple structural conventions but does not follow strict syntax of any programming language. It is meant to be read and understood by humans before actual coding.

Pseudo-code conventions:

- *Assignment statement:* `variable ← expression` or `variable = expression` Example: `total ← a + b`
- *Input/Output:* `READ x`, `PRINT result`
- *Basic control structures:*
- Sequence: statements executed one after another.
- Selection: `IF condition THEN ... ELSE ... ENDIF`
- Iteration: `WHILE condition DO ... ENDWHILE`, `FOR i = 1 TO n DO ... ENDFOR`

Example — Pseudo-code to add two numbers:

```
START
  READ a, b
  sum ← a + b
  PRINT sum
STOP
```

1.2 Algorithmic Problems

(i) Algorithm to solve a simple arithmetic expression

```
START
  READ a, b
  sum ← a + b
  diff ← a - b
  prod ← a * b
  IF b ≠ 0 THEN
    quot ← a / b
  PRINT sum, diff, prod, quot
STOP
```

(ii) Algorithm to find the greatest of three numbers

```

START
  READ a, b, c
  IF a > b THEN
    IF a > c THEN
      PRINT "a is greatest"
    ELSE
      PRINT "c is greatest"
  ELSE
    IF b > c THEN
      PRINT "b is greatest"
    ELSE
      PRINT "c is greatest"
STOP

```

(iii) Algorithm to determine whether a number is even or odd

```

START
  READ n
  IF n MOD 2 = 0 THEN
    PRINT "Even"
  ELSE
    PRINT "Odd"
STOP

```

(iv) Algorithm to determine whether a number is prime

```

START
  READ n
  count ← 0
  FOR i = 2 TO n-1 DO
    IF n MOD i = 0 THEN
      count ← count + 1
  IF count = 0 AND n > 1 THEN
    PRINT "Prime"
  ELSE
    PRINT "Not Prime"
STOP

```

1.3 Flowchart

A **flowchart** is a pictorial/graphical representation of an algorithm using standard geometrical symbols connected by arrows that show the flow of control/logic.

Symbols of Flowchart:

Symbol	Name	Purpose
Oval/Ellipse	Terminal	Start/Stop of the flowchart
Parallelogram	Input/Output	Used for READ/PRINT statements
Rectangle	Process	Used for calculations/assignments
Diamond	Decision	Used for condition checking (yes/no, true/false)
Circle	Connector	Used to connect flow when flowchart spans pages
Arrow	Flow line	Shows the direction of logic flow

Guidelines for preparing a flowchart:

1. Use standard symbols only.
2. Flowchart should have a clear start and stop point.

3. Flow direction should normally be top to bottom and left to right.
4. Decision boxes must have exactly two (or more) clearly labeled exits (e.g., Yes/No).
5. Keep the flowchart simple, neat, and avoid unnecessary crossing of lines.
6. Use connectors when the flowchart is long and spans multiple pages.
7. Every loop must have a proper exit condition to avoid infinite looping.

Example — Flowchart logic to check even/odd number (described): Start → Input n → Decision: Is n MOD 2 = 0? → Yes: Print “Even” → No: Print “Odd” → Stop.

Unit 2.0 — Basics of C Programming

2.1 Introduction to C

History of C: C was developed by **Dennis Ritchie** at Bell Laboratories in **1972**. It evolved from earlier languages ALGOL → BCPL → B → C. C was originally created to re-write the UNIX operating system. It is a **middle-level, structured, procedural** programming language that combines the efficiency of assembly language with the readability of high-level languages.

General structure of a C program:

```
#include <stdio.h>      /* Header file inclusion */

/* Global declarations, if any */

int main()              /* main function - entry point */
{
    /* Declaration part */
    /* Executable statements */
    return 0;
}
```

- **Header files:** Files with `.h` extension (e.g., `stdio.h`, `conio.h`, `math.h`) that contain pre-defined function prototypes and macros. Included using `#include`.
- **main() function:** Every C program must have exactly one `main()` function — it is the starting point of program execution.

2.2 Data Concepts

Character set: The basic set of characters a C compiler can recognize — alphabets (A-Z, a-z), digits (0-9), special symbols (+, -, *, /, {, }, :, etc.) and white spaces.

Tokens: The smallest individual unit in a C program. Types: keywords, identifiers, constants, strings, operators.

Keywords: Reserved words with predefined meaning, cannot be used as identifiers. Example: `int`, `float`, `char`, `if`, `else`, `for`, `while`, `return`, `struct`, `void` (32 keywords in standard C).

Identifiers: Names given to variables, functions, arrays, etc. Rules: Must start with a letter or underscore, can contain letters/digits/underscore, cannot be a keyword, case-sensitive. Example: `total`, `_sum`, `marks1`

Variables: A named memory location whose value can change during program execution. Example: `int age;`

Constants: Fixed values that do not change during execution. - Integer constant: `10`, `-25` - Real/Float constant: `3.14`, `-0.5` - Character constant: `'A'` - String constant: `"Hello"`

Data types in C:

Type	Keyword	Size (typical)	Example
Integer	<code>int</code>	2/4 bytes	<code>int x = 10;</code>
Character	<code>char</code>	1 byte	<code>char c = 'A';</code>
Floating point	<code>float</code>	4 bytes	<code>float f = 5.5;</code>
Double precision	<code>double</code>	8 bytes	<code>double d = 5.55555;</code>
Valueless	<code>void</code>	—	used for function return type

C Operators:

--	--	--

Category	Operators	Example
Arithmetic	+ - * / %	a + b , a % b
Relational	< > <= >= == !=	a > b
Logical	&& \ \ !	(a>b) && (b>c)
Assignment	= += -= *= /=	a += 5;
Increment/Decrement	++ --	a++; --b;
Bitwise	& \ ^ ~ << >>	a & b
Conditional (ternary)	?:	max = (a>b) ? a : b;

Arithmetic expression: A combination of variables, constants and arithmetic operators. Example: `area = 3.14 * r * r;`

Declaring variables:

```
int a, b, c;
float price = 99.5;
char grade = 'A';
```

Data type conversion: - *Implicit (automatic) conversion:* Compiler automatically converts lower type to higher type in mixed expressions. Example: `int a = 5; float b = 2.0; float c = a + b; // a is converted to float` - *Explicit conversion (Type casting):* Programmer forces conversion. Example: `float avg = (float) sum / n;`

2.3 Basic Input Output

printf() : Used to display/output data on the screen.

```
printf("Value of a = %d", a);
```

Common format specifiers: `%d` (int), `%f` (float), `%c` (char), `%s` (string), `%lf` (double).

scanf() : Used to read input from the keyboard.

```
scanf("%d", &a); // & is the address-of operator
```

Character I/O statements: `getchar()` reads a single character; `putchar()` displays a single character.

```
char ch;
ch = getchar();
putchar(ch);
```

Input/Output formatting: Field width and precision can be specified.

```
printf("%5d", 23); // prints with field width 5
printf("%.2f", 3.14159); // prints 3.14 (2 decimal places)
```

Use of comments: Comments are non-executable lines used for documentation, ignored by the compiler.

```
// single line comment
/* multi
   line comment */
```

Example program:

```
#include <stdio.h>
int main()
```

```
{  
    int a, b, sum;  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);  
    sum = a + b;  
    printf("Sum = %d", sum);  
    return 0;  
}
```

Unit 3.0 — Decision Making and Branching

3.1 Decision Making and Branching

Relational operators: Used to compare two values: `<` `>` `<=` `>=` `==` `!=`

Logical operators: Used to combine conditions: `&&` (AND), `||` (OR), `!` (NOT)

if statement:

```
if (marks >= 40)
    printf("Pass");
```

if-else statement:

```
if (marks >= 40)
    printf("Pass");
else
    printf("Fail");
```

Nested if-else: An if-else statement inside another if or else block.

```
if (a > b) {
    if (a > c)
        printf("a is greatest");
    else
        printf("c is greatest");
}
```

if-else ladder: Used when multiple conditions are to be checked sequentially.

```
if (marks >= 90)
    printf("Grade A");
else if (marks >= 75)
    printf("Grade B");
else if (marks >= 60)
    printf("Grade C");
else
    printf("Grade D");
```

The switch statement: Used for multi-way branching based on the value of an expression.

```
switch (day) {
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    default: printf("Invalid day");
}
```

Note: `break` is used to exit the switch after a matching case; without it, execution falls through to the next case.

3.2 Looping

while loop: Entry-controlled loop; condition is tested before the body executes.

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

do...while loop: Exit-controlled loop; body executes at least once, condition tested after the body.

```
int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);
```

for loop: Combines initialization, condition, and increment/decrement in one line.

```
for (int i = 1; i <= 5; i++)
    printf("%d ", i);
```

goto statement: Used for unconditional jump to a labeled statement (use sparingly, reduces readability).

```
int i = 1;
label:
    printf("%d ", i);
    i++;
    if (i <= 5) goto label;
```

break statement: Terminates the loop or switch immediately.

```
for (i = 1; i <= 10; i++) {
    if (i == 5) break;
    printf("%d ", i);
}
```

continue statement: Skips the current iteration and moves to the next iteration of the loop.

```
for (i = 1; i <= 5; i++) {
    if (i == 3) continue;
    printf("%d ", i);
}
```

Unit 4.0 — Array and Structure

4.1 Arrays

Characteristics of an array: An array is a collection of elements of the **same data type** stored in **contiguous memory locations**, accessed using a common name and an index (subscript).

One-dimensional array:

```
int marks[5];           // declaration
int marks[5] = {90,85,78,92,88}; // declaration with initialization
printf("%d", marks[2]); // accessing third element (78)
```

Two-dimensional array: Represents data in rows and columns (matrix form).

```
int matrix[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
printf("%d", matrix[1][2]); // prints 6
```

Array declaration and initialization:

```
int arr[5];           // uninitialized
int arr[5] = {1,2,3,4,5}; // fully initialized
int arr[] = {1,2,3};   // size inferred
```

4.2 Array of Characters and Operations

Array of characters (String): A string in C is an array of characters terminated by a null character `'\0'`.

```
char name[20] = "Computer";
```

Operations on array: Traversal, insertion, deletion, searching, sorting.

```
// Sum of array elements
int sum = 0;
for (int i = 0; i < 5; i++)
    sum += arr[i];
```

Character and String I/O:

```
char str[20];
gets(str);           // reads a line (legacy, unsafe)
fgets(str, 20, stdin); // safer alternative
puts(str);           // prints a string
printf("%s", str);
scanf("%s", str);    // reads a word (no spaces)
```

4.3 Structures

Introduction and features of Structures: A structure is a user-defined data type that groups variables of **different data types** under a single name.

Declaration and initialization:

```
struct Student {
    int roll;
    char name[20];
    float marks;
};
```

```
struct Student s1 = {1, "Aman", 89.5};  
printf("%s scored %f", s1.name, s1.marks);
```

typedef : Used to create a new name (alias) for an existing data type.

```
typedef struct Student STU;  
STU s2;
```

Enumerated Data Type (enum): A user-defined type consisting of integral named constants.

```
enum week {MON, TUE, WED, THU, FRI, SAT, SUN};  
enum week today = WED;  
printf("%d", today); // prints 2
```

Using structures in a C program:

```
struct Book {  
    int acc_no;  
    char title[30];  
    char author[20];  
    float price;  
};  
  
int main() {  
    struct Book b1 = {101, "C Programming", "Balaguruswamy", 350.0};  
    printf("%d %s %s %.2f", b1.acc_no, b1.title, b1.author, b1.price);  
    return 0;  
}
```

Unit 5.0 — Concept and Need of Functions

5.1 Library Functions

A **library function** is a pre-defined, built-in function provided by C compilers in header files; the programmer simply calls it without writing its code.

Math functions (from `math.h`):

```
sqrt(25.0);    // 5.0
pow(2,3);      // 8.0
abs(-5);       // 5
ceil(4.3);     // 5.0
floor(4.7);    // 4.0
```

String handling functions (from `string.h`):

```
strlen(str);    // length of string
strcpy(dest, src); // copy string
strcat(s1, s2); // concatenate
strcmp(s1, s2); // compare (0 if equal)
strrev(str);    // reverse string
```

Other miscellaneous functions: `rand()`, `clrscr()`, `exit()`, `system()`.

5.2 User-defined Functions

Need of functions: Modular programming, code reusability, easier debugging, reduced code redundancy.

Writing user-defined functions:

```
int add(int a, int b)    // function definition
{
    return a + b;
}

int main()
{
    int result = add(5, 3); // function call
    printf("%d", result);
    return 0;
}
```

Scope of variables: - *Local variable:* Declared inside a function; accessible only within that function. -

Global variable: Declared outside all functions; accessible throughout the program.

Parameter passing:

- **Call by value:** A copy of the actual argument is passed; changes inside the function do not affect the original variable.

```
void change(int x) { x = 100; }
int a = 10;
change(a);    // a remains 10
```

- **Call by reference:** The address of the variable is passed; changes inside the function affect the original variable.

```
void change(int *x) { *x = 100; }
int a = 10;
change(&a);    // a becomes 100
```

5.3 Recursive Functions

A **recursive function** is a function that calls itself to solve a smaller instance of the same problem, until a base condition is met.

Example — Factorial using recursion:

```
int factorial(int n)
{
    if (n == 0)
        return 1;           // base condition
    else
        return n * factorial(n-1); // recursive call
}
```

Example — GCD using a function:

```
int gcd(int a, int b)
{
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
```

Unit 6.0 — Pointers

6.1 Concepts of Pointers

A **pointer** is a variable that stores the **memory address** of another variable.

Declaring a pointer:

```
int a = 10;
int *p;    // p is a pointer to int
p = &a;    // p holds address of a
```

Initializing and accessing:

```
printf("%d", *p); // *p gives the value at address stored in p (dereferencing)
printf("%p", p);  // prints the address
```

Pointer arithmetic: Arithmetic operations on pointers move the pointer by (value) * size_of_data_type bytes.

```
int arr[5] = {10,20,30,40,50};
int *p = arr;
p++;      // now points to arr[1]
printf("%d", *p); // prints 20
```

6.2 Handling Arrays, Functions and Structures using Pointers

Handling arrays using pointers:

```
int arr[5] = {1,2,3,4,5};
int *p = arr;
for (int i = 0; i < 5; i++)
    printf("%d ", *(p+i));    // accessing array elements via pointer
```

Handling functions using pointers (pointer as function argument — call by reference):

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("%d %d", x, y);    // 10 5
}
```

Handling structures using pointers:

```
struct Student {
    int roll;
    char name[20];
};

int main() {
    struct Student s1 = {1, "Aman"};
    struct Student *ptr = &s1;
    printf("%d %s", ptr->roll, ptr->name); // arrow operator used with structure pointers
    return 0;
}
```

Key points: - * (dereference operator) gives the value stored at an address. - & (address-of operator) gives the address of a variable. - -> (arrow operator) is used to access structure members through a pointer. - Pointer to array elements: *(p+i) is equivalent to p[i] .